

Taller práctico: del gráfico a la lógica

Ordenamiento por selección

Estructura de datos

Hecho por: Ashley Dayanne Rodríguez Acosta

FASE I: Exploración y Visualización Dinámica

1. Preparación de la Visualización

Recurso	Detalle
Código/Pseudocódigo	Código de ordenamiento por selección en lenguaje de programación Python
Herramienta	Visualizer
Datos	Entrada: [6, 3, 5, 2, 4]

2. Observación Activa

1. ¿Cuál es el patrón principal de movimiento o de acción? *El algoritmo recorre el arreglo buscando el valor mínimo y lo coloca en la primera posición libre. Visualmente se observa cómo un elemento se “destaca” (coloreado) mientras se compara con los demás, y luego se intercambia su posición con el menor encontrado*
2. ¿Cómo “detecta” visualmente el algoritmo que ha terminado su trabajo? *Al final, todos los elementos cambian de color (generalmente a verde o azul), indicando que el arreglo ya está completamente ordenado y no quedan comparaciones pendientes.*
3. Si el algoritmo tiene una fase de "división" y otra de "unión", ¿cómo se distingue el cambio? *En este algoritmo no hay una división explícita. Sin embargo, se distingue una transición visual entre la búsqueda del mínimo (comparaciones en color resaltado) y el intercambio de posiciones (movimiento o cambio de color de dos elementos).*

FASE II: Prueba de Escritorio (Trazado de la Lógica)

1. Caso de Prueba y Planificación

Elemento	Descripción de la tarea
Caso de prueba	lista = [5, 2, 4, 1]
Tabla de trazado	Variables Críticas: arreglo, i, j, min
	Condición: $A[j] < A[\text{min}]$

2. Ejecución Manual (Simulación paso a paso)

Paso	i	j	min	Condición	Resultado
1	-	-	-	Inicio	[5, 2, 4, 1]
2	0	1	0	$\text{lista}[1] < \text{lista}[0]? \rightarrow \text{True}$ $\text{min}=1$	[5, 2, 4, 1]
3	0	2	1	$\text{lista}[2] < \text{lista}[1]? \text{ False}$	[5, 2, 4, 1]
4	0	3	1	$A[3] < A[1]? \rightarrow \text{True}$ $\text{min} = 3$	[5, 2, 4, 1]

FASE III: Validación y Reflexión

1. Validación de la Lógica

Tras ejecutar el mismo caso de prueba en Algorithm Visualizer, se comprobó que las comparaciones y los intercambios coincidieron con las filas de la tabla.

El algoritmo realizó exactamente tres intercambios, como se registró manualmente.

2. Reporte de Conclusión

- **Conexión Visual-Lógica:**

En la visualización, cuando un elemento cambiaba de color y se movía a la primera posición (por ejemplo, el número 1 moviéndose al inicio), eso correspondía en el código al momento donde la condición `A[j] < A[min]` era verdadera y se ejecutaba la línea `intercambiar A[i] y A[min]`.

- **Aporte de la Prueba de Escritorio:**

La prueba de escritorio ayudó a comprender cómo cambia la variable `min` dentro del ciclo interno y cómo el algoritmo “recuerda” la posición del menor elemento sin hacer intercambios inmediatos.

Esto permitió visualizar mejor que el intercambio sólo ocurre una vez por iteración del ciclo externo, lo cual explica por qué el algoritmo es menos eficiente que otros, pero más fácil de entender.